

שנקר ביי"ס גבוה להנדסה ולעיצוב
החוג להנדסת תוכנה
קורס 'תכנות מערכות' (קורס מספר 3503820)
תשפ"ו – סמסטר קיץ

מרצה: יצחק נודלר

תרגיל בית מס' 2 -- גרסה 'מאתגרת'

מועד הגשה: 09-09-2026

הנחיות:
מתוארות בהמשך.

מטלת תכנות: מימוש מנגנון Crash Recovery מקבילי (Multi-Threaded WAL Engine) ב-C

הערה כללית: 'חוט' = thread

1. **רקע תיאורטי: עקרונות ACID ומנגנון WAL**
במערכות מסדי נתונים מודרניות (כדוגמת PostgreSQL, MySQL או SQLite), מנוע הטרנזקציות מחויב לשמור על ארבעה עקרונות יסוד המכונים **ACID**:

- **Atomicity (אטומיות - "הכל או כלום")**: טרנזקציה היא יחידת עבודה שלא ניתן לחלק. או שכל הפעולות הכלולות בה מתבצעות ונרשמות בהצלחה, או שאף אחת מהן לא מתקבלת. אם הטרנזקציה נקטעת באמצע (עקב שגיאה או קריסת מערכת), יש לבטל לחלוטין את כל עדכוני הביניים שביצעה.
- **Consistency (עקביות)**: הטרנזקציה מעבירה את מסד הנתונים ממצב תקין אחד למצב תקין אחר, תוך שמירה על כל חוקי ואילוצי המערכת.
- **Isolation (בידוד)**: ביצוע במקביל של מספר טרנזקציות על ידי חוטים (Threads) שונים חייב להביא לתוצאה זהה כאילו הן הורצו באופן סדרתי (Serial), מבלי שטרנזקציה אחת תחשוף מידע חלקי או לא יציב לטרנזקציה אחרת.
- **Durability (עמידות)**: ברגע שטרנזקציה קיבלה אישור שהסתיימה בהצלחה (COMMIT), כל השינויים שביצעה נשמרים באופן קבוע בזיכרון לא-נדיף (דיסק), וישרדו גם קריסה מלאה של המערכת או נפילת מתח.

מנגנון Write-Ahead Logging (WAL)

כדי להבטיח Atomicity ו-Durability ביעילות מבלי לכתוב לדיסק את כל דפי הזיכרון בכל פקודה, המערכת משתמשת ביומן WAL.

כל שינוי נרשם ראשית ביומן ה-WAL בדיסק. אם המערכת קורסת באמצע הריצה, זיכרון ה-RAM מתאפס. עם העלייה מחדש, ה-Recovery Engine קורא את קובץ ה-WAL מהתחלה ועד נקודת הקריסה, ומבצע Undo לכל הטרנזקציות שהיו באמצע הריצה (ACTIVE) ברגע הקריסה ולא הספיקו להגיע ל-COMMIT או ABORT.

2. מפרט קבצי הקלט והפלט

קובץ הקלט: wal_input.txt

הקובץ מכיל שורות טקסט המייצגות אירועים כרונולוגיים שנוצרו על ידי חוטים שונים במקביל. כל השורות מיושרות לשמאל.

פורמטים אפשריים לשורה בקובץ:

- BEGIN <ID> TX – פתיחת טרנזקציה.
- <VAL2>: NEW <VAL1>: OLD <KEY> UPDATE <ID> TX – עדכון ערך של מפתח/דף.
- COMMIT <ID> TX – סיום מוצלח של טרנזקציה.
- ABORT <ID> TX – ביטול יזום של טרנזקציה.
- === CRASH === – נקודת הקריסה. אין לקרוא שורות מעבר לשורה זו!

קובץ הפלט: accounts.txt

קובץ טקסט שייווצר על ידי התוכנית שלכם בסיום תהליך ה-Recovery. הקובץ יכיל את המצב הסופי המשוחזר של כל המפתחות (page_X) שהופיעו בלוג.

פורמט הפלט הנדרש:

- מפתח וערך בכל שורה, מופרדים על ידי נקודתיים ורווח: <KEY> : <VALUE>
- כל השורות מיושרות לשמאל וממוינות לפי סדר אלפביתי של המפתחות.

3. ארכיטקטורה ודרישת מקביליות (Multi-Threading)
כדי לבצע את שלב התאוששות ה-Undo ביעילות, התוכנית שלכם תשתמש בספריית POSIX Threads (pthreads) ובמנגנוני סנכרון:

1. חוט ראשי (Main Thread - Analysis Phase):

- סורק את הקובץ wal_input.txt מתחילתו ועד לשורה `=== CRASH ===`.
- בונה בזיכרון את טבלת המצב העדכנית של ה-Pages ואת היסטוריית הלוגים.
- מזהה את K הטרנזקציות שנותרו במצב ACTIVE ברגע הקריסה.

2. חוטי עבודה (Worker Threads - Parallel Undo Phase):

- ה-Main Thread מפיק K חוטי עבודה מקביליים באמצעות `pthread_create`, כאשר כל חוט אחראי באופן בלעדי על ביצוע Rollback לטרנזקציה אקטיבית אחת.
- כל Worker Thread סורק את היסטוריית השינויים של הטרנזקציה שלו מהסוף להתחלה ומחזיר את ה-page_X הרלוונטיים לערך ה-OLD שלהם.

3. סנכרון ואיזורי מגן (Mutexes & Thread Safety):

- מכיוון שחוסים שונים עשויים לבצע Undo במקביל ולגשת לאותם דפים (page_X) בטבלת הזיכרון המשותפת, חובה למנוע Race Conditions.
- יש להשתמש ב-`pthread_mutex_t` כדי לנועל את הגישה לטבלת ה-Pages בזיכרון בזמן עדכון ערך.

4. סנכרון סיום:

- ה-Main Thread ימתין לסיום כל חוטי העבודה באמצעות `pthread_join`, ורק לאחר מכן ימייץ את הנתונים ויכתוב את הקובץ הסופי `accounts.txt`.

4. מבני נתונים מומלצים ב-C

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <pthread.h>

#define MAX_LINE_LEN 256

typedef enum { TX_INACTIVE, TX_ACTIVE, TX_COMMITTED, TX_ABORTED }
TxState;

// Mutex טבלת דפים משותפת המוגנת על ידי
typedef struct {
    char key[64];
    int value;
} Page;

typedef struct {
    Page pages[1000];
    int page_count;
    pthread_mutex_t lock; // מנעול לסנכרון גישה חוטים
} SharedDatabase;

// רשומת לוג לצורך ביטול
typedef struct {
    int tx_id;
    char key[64];
    int old_val;
    int new_val;
} LogEntry;

// Worker Thread ארגומנט שמועבר לכל
typedef struct {
    int target_tx_id;
    LogEntry* log_history;
    int log_size;
    SharedDatabase* db;
} ThreadArgs;
```

5. אלגוריתם הביצוע - שלב אחר שלב

שלב א': סריקה קדימה (Forward Pass)

- קריאת wal_input.txt שורה-אחר-שורה עד === CRASH ===.
- עדכון ה-SharedDatabase בערכי ה-NEW ושמירת אירועי ה-UPDATE במערך היסטוריה.
- מעקב אחר מצב הטרנזקציות (ABORTED, COMMITTED, ACTIVE).

שלב ב': היערכות ל-Rollback מקבילי

- זיהוי כל ה-tx_id שנשארו בסטטוס ACTIVE.
- אתחול מנעול הסנכרון pthread_mutex_init(&db.lock, NULL).

שלב ג': הרצת חוטי העבודה (Parallel Undo)

- הקצאת מערך של pthread_t בגודל מספר הטרנזקציות האקטיביות.
- הזנקת החוטים בלולאה בעזרת pthread_create, כאשר כל חוט מקבל את ה-tx_id שלו.
- **בתוך פונקציית החוט:** החוט עובר על היסטוריית הלוג מהסוף להתחלה, מוצא שינויים השייכים ל-tx_id שלו, נועל את ה-Mutex, משחזר את הערך ל-OLD, ומשחרר את ה-Mutex.

שלב ד': סגירה וכתובת פלט

- ה-Main Thread מבצע pthread_join לכל החוטים ומשמיד את ה-Mutex (pthread_mutex_destroy).
- מיון טבלת ה-Pages לפי שם המפתח (למשל בעזרת qsort ו-stremp).
- כתיבת התוצאה המשוחזרת לקובץ accounts.txt.

6. דרישות תיעוד והסבר (מפרט חובה)

על מנת לוודא שכל גורם חיצוני שקורא את ההגשה שלכם יוכל להבין במלואו מה התוכנית עושה וכיצד היא עושה זאת, עליכם לעמוד בשתי הדרישות הבאות:

דרישה 1: תיעוד נרחב בגוף הקוד (In-Code Documentation)

- **תיאור קבצים ופונקציות:** בראש קובץ הקוד ובראש כל פונקציה יש לכתוב בלוק הערה המפרט את תפקידה, הארגומנטים שהיא מקבלת, ערכי החזרה, והנחות יסוד (Assumptions).
- **הסבר שורה-אחר-שורה/בלוקים:** יש לצרף הערות מפורטות לצד קטעי קוד מורכבים (למשל: לוגיקת ה-Parsing, ניהול הזיכרון הדינמי, הקצאת ה-pthreads, ונעילת/שחרור ה-Mutexes).
- **הסבר ה-Race Conditions:** במקומות שבהם משתמשים ב-pthread_mutex_lock, יש להסביר בהערה בגוף הקוד מאיזה Race Condition המנעול מגן בדיוק.

דרישה 2: מסמך הסבר ארכיטקטורה מקיף (DESIGN.txt / DESIGN.pdf) עליכם לצרף להגשה מסמך נפרד ומפורט הכולל את הסעיפים הבאים:

1. **סקירה כללית (Overview):** הסבר נרחב על זרימת התוכנית (Execution Flow) מרגע טעינת wal_input.txt ועד כתיבת accounts.txt.
2. **מבני הנתונים (Data Structures):** פירוט כל ה-structs וה-enums שגדרתם, מטרתם, ומדוע נבחרו (למשל: שימוש במערך דינמי מול רשימה מקושרת).
3. **מנגנון המקביליות והסנכרון (Synchronization Design & Concurrency):**
 - הסבר מפורט על אופן החלוקה של הטרנזקציות האקטיביות ל-Worker Threads.
 - ניתוח של איזורי המגן (Critical Sections) בתוכנית והוכחה כיצד המנעולים שהטמעתם מונעים מ-Race Conditions ו-Deadlocks.
4. **טיפול בשגיאות ומקרי קצה (Error Handling & Edge Cases):** הסבר כיצד התוכנית מתמודדת עם קבצים פגומים, כשלים בהקצאת זיכרון, או כשלים ביצירת חוטים.

7. טיפול מקרי קצה ושגיאות

- **אי קיום קובץ קלט:** הדפסת הודעת שגיאה ל-stderr ויציאה ב-exit(1).
- **כשל בהקצאת חוטים / מנעולים:** בדיקת ערך החזרה של pthread_create ו-pthread_mutex_lock.
- **שורות ריקות:** התעלמות מרווחים ושורות ריקות בתוך הקובץ.

הגשה:

את ה-source file ואת מסמך הסבר הארכיטקטורה יש להעלות לאתר הקורס.